

Focusing on the Planning Module: A Step Toward Three-Component MLS Analysis

Rashedul Arefin Ifty

I. INTRODUCTION

The previous meeting resolved two foundational questions: how many retraining policies exist for a two-component MLS (48 valid policies), and how to evaluate the planning module (C3) using a three-tier framework. With those resolved, the natural next step is to zoom into C3 itself before building any formal model around it.

This report does not attempt to build the full three-component SMP. Instead, it takes one concrete step forward: understanding the planning module deeply enough to know what to measure, which model to use, how it receives input from the pipeline, and what we expect to happen when that input is corrupted by upstream retraining. This understanding is the necessary foundation before any modeling work begins.

The goal of this report is to answer: *what is the planning module, how does it work in our pipeline, what happens when upstream data is faulty, and what small experiment can confirm this before the next meeting?*

II. WHAT IS THE PLANNING MODULE?

In the autonomous driving pipeline, planning is the final component. It takes as input the predictions of how other agents (cars, pedestrians, cyclists) will move, and produces as output a trajectory—a sequence of positions and velocities that the ego-vehicle will follow over the next few seconds.

The planner does not see the raw sensor data. It only sees what the upstream components give it:

- From C1 (Perception): a list of detected objects with their current positions and sizes.
- From C2 (Prediction): a forecast of where each detected object will be in the next 2–6 seconds.

The planner uses this information to answer one question: *given where everything is and where everything is going, what path should this vehicle take?*

This dependency is what makes planning vulnerable. If C1 misses an object, or if C2 predicts its trajectory incorrectly, the planner receives a false picture of the world. It will produce a plan that is optimal for the world it *thinks* it is in—but dangerous for the world it *actually* is in.

III. WHICH PLANNING MODEL TO USE

For this research, we select **PDM-Closed** [3] as the planning module C3. There are three reasons for this choice.

Modularity. PDM-Closed is a standalone module that takes predicted agent trajectories as input and produces a planned

ego-trajectory as output. It does not jointly train with perception or prediction. This is essential for our research: we need C3 to be independently evaluable so we can isolate its own performance from upstream faults.

nuScenes benchmark compatibility. PDM-Closed is one of the top-performing models on the official nuScenes planning benchmark [2]. Using it means our results are directly comparable to the community baseline, which gives the research credibility.

Interpretability. PDM-Closed uses a scoring function over candidate trajectories. When its output degrades, the reason is traceable—the input predictions changed. This is important for our analysis: we want to confirm that degradation in C3 comes from upstream, not from C3 itself.

The evaluation metrics for C3 are:

- **L2 displacement error:** distance between the planned trajectory and the human driver’s trajectory at 1 s, 2 s, and 3 s horizons. This is the primary metric.
- **Collision rate:** percentage of scenarios where the planned trajectory intersects with another agent. This is the primary safety metric.

IV. HOW THE PIPELINE FEEDS INTO PLANNING

Figure 1 illustrates the data flow from raw sensor input to the final plan.

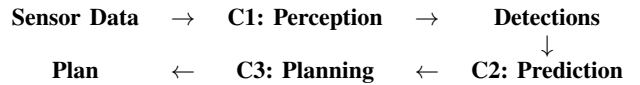


Fig. 1. Data flow in the three-component autonomous driving pipeline.

Each transition in this pipeline is a potential point of failure. When C1 is retrained and its output distribution changes, C2 receives inputs that look different from what it was trained on. C2’s predictions then shift—not because C2 was retrained, but because its inputs changed. C3 then receives shifted predictions and produces a plan based on an inaccurate picture of the world.

This is the cascade: a single retraining event at C1 propagates through C2 and arrives at C3 as degraded input. The damage is invisible if you only look at C1’s own metric—its detection accuracy may have improved. But the system-level planning output may be worse.

V. WHAT HAPPENS WITH FAULTY UPSTREAM DATA

To understand the risk to planning, we consider three concrete scenarios that can occur after upstream retraining.

A. Missing Detections

If C1 after retraining misses objects it previously detected—for example, pedestrians at long range—C2 never receives those objects as input and therefore never predicts their trajectories. C3 plans as if those agents do not exist. The result is a trajectory that passes through the space those agents occupy.

This is the most dangerous failure mode. It is also silent: C1’s overall mAP may improve (it now detects short-range objects better), while its recall on long-range pedestrians drops. The aggregate metric improves; the safety-critical sub-task fails.

B. Shifted Prediction Distributions

Even if C1 still detects all objects, a change in its output format—slightly different bounding box sizes, different confidence scores, slightly shifted center positions—can push C2’s inputs outside the distribution it was trained on. C2’s trajectory forecasts then become less accurate. This is a subtler failure: C2’s own minADE score (measured with ground-truth detections as input) would be unchanged, but its real-world prediction quality in the live pipeline degrades.

This is precisely the scenario the three-tier evaluation framework from Meeting 2 was designed to detect. Tier 2 of that framework—evaluating C2 with ground-truth detections versus live C1 detections—will quantify this shift directly.

C. Compounded Error at Planning

When both effects occur together—missed detections and shifted prediction distributions—C3 faces a compounded problem. Its input contains fewer agents, and the predictions for the agents it does see are less reliable. The planning L2 error increases, and the collision rate rises. This is what we expect to observe in Experiment E1 (retrain C1 only) from the experimental design in Meeting 1.

VI. PROPOSED INITIAL EXPERIMENT

Before building the full SMP model, one targeted experiment can confirm that cascade degradation actually reaches C3. This experiment is small enough to complete before the next meeting and produces exactly the empirical evidence needed to justify the modeling work.

A. Setup

Train all three components independently on the nuScenes Boston split:

- C1: CenterPoint [5] for 3D object detection.
- C2: A simple MTP (Multiple Trajectory Prediction) baseline, which is lightweight and fast to train.
- C3: PDM-Closed, using C2’s predictions as input.

Record the baseline performance of each component and the end-to-end pipeline on the nuScenes validation set.

TABLE I
MEASUREMENTS FOR THE INITIAL CASCADE EXPERIMENT

Component	Metric	Expected Change
C1	mAP on Singapore data	Improves (this is what we retrained for)
C2 (isolated)	minADE with ground-truth detections	Unchanged (C2 not retrained)
C2 (pipeline)	minADE with live C1 output	May worsen (input distribution shifted)
C3 (isolated)	L2 error with ground-truth predictions	Unchanged (C3 not retrained)
C3 (pipeline)	L2 error with live C2 output	May worsen (cascade effect)

B. The Critical Test

Retrain C1 only using the Singapore split data, simulating a domain shift adaptation. Do not retrain C2 or C3. Then measure:

The key observation is the gap between the isolated and pipeline measurements for C2 and C3. If C3 (pipeline) is worse than C3 (isolated), cascade degradation has been empirically confirmed.

C. What a Positive Result Means

If the experiment shows that retraining C1 degrades C3’s pipeline performance while C3’s isolated performance is unchanged, it confirms three things:

- 1) Cascade entanglement exists in this three-component system.
- 2) The fault is upstream, not in C3 itself.
- 3) The three-tier framework correctly diagnoses the source.

This result justifies all subsequent modeling work—the 8-state SMP, the cascade coverage factor c_{cas} , and the policy analysis—by grounding them in observed real-world behavior.

VII. WHAT COMES NEXT

This experiment is a starting point, not the full contribution. Once cascade degradation is confirmed empirically, the following steps will be taken in subsequent meetings:

Step 1 — Quantify coupling strength. Compute the error propagation ratio $\rho_{1 \rightarrow 3}$ from the experimental results. This tells us how strongly C1 retraining affects C3 output and feeds directly into the SMP coverage factors.

Step 2 — Run all seven experiments. Complete the full set of retraining subsets from Meeting 1 (E1 through E7) to get a complete picture of how different combinations of retraining affect the pipeline.

Step 3 — Build the 8-state SMP. Use the experimental measurements to calibrate the full three-component SMP model, replacing synthetic parameters with real data.

Step 4 — Policy analysis. Evaluate the policy families (including the two new C3-focused families) against the calibrated SMP to find the Pareto-optimal retraining strategy.

VIII. CONCLUSION

This report takes one focused step forward from Meeting 2: understanding the planning module in enough detail to design a meaningful first experiment. The key insight is that C3 is uniquely vulnerable—it cannot distinguish between “the world is safe” and “the world looks safe because upstream models gave me incomplete information.” This makes it the most important component to protect, and the most difficult to evaluate correctly.

The proposed initial experiment is simple, executable before the next meeting, and produces a clear yes-or-no answer to the most important question in this project: *does retraining C1 actually degrade C3?* Everything else builds from that answer.

REFERENCES

- [1] Z. Wang and F. Machida, “Exploiting the availability-continuity trade-off in imperfect retraining of machine learning systems,” in *Proc. 36th IEEE Int. Symp. Software Reliability Engineering (ISSRE)*, 2025.
- [2] H. Caesar *et al.*, “nuScenes: A multimodal dataset for autonomous driving,” in *Proc. IEEE/CVF CVPR*, 2020, pp. 11621–11631.
- [3] D. Dauner, M. Hallgarten, A. Geiger, and K. Chitta, “Parting with misconceptions about learning-based vehicle motion planning,” in *Proc. CoRL*, 2023.
- [4] T. Yuan, Z. Kitani, K. Kitani, and J. Shi, “AgentFormer: Agent-aware transformers for socio-temporal multi-agent forecasting,” in *Proc. IEEE/CVF ICCV*, 2021.
- [5] T. Yin, X. Zhou, and P. Krähenbühl, “Center-based 3D object detection and tracking,” in *Proc. IEEE/CVF CVPR*, 2021.